

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Paralelní Programování v jazyku Go



2026

Vedoucí práce:
Mgr. Jan Laštovička, Ph.D.

Richard Bebjak

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor:	Richard Bebjak
Název práce:	Paralelní Programování v jazyku Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Jan Laštovička, Ph.D.
Počet stran:	28
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Richard Bebjak
Title:	Parallel programming in language Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Jan Laštovička, Ph.D.
Page count:	28
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Bakalárska práca sa zaoberá paralelným programovaním v programovacom jazyku Go. Teoretická časť práce predstavuje základné princípy súbežného programovania s dôrazom na jeho využitie pri paralelnom vykonávaní úloh, architektúru jazyka Go a vstavané mechanizmy podporujúce súbežné programovanie. Súčasťou práce je návrh algoritmov využívajúcich tieto mechanizmy a analýza ich správnosti prostredníctvom dôkazov. Praktická časť je zameraná na návrh a implementáciu prototypu chat servera, na ktorom sú demonštrované možnosti súbežného programovania v jazyku Go a využitie navrhnutých algoritmov v praxi.

Synopsis

This bachelor's thesis focuses on parallel programming in the Go programming language. The theoretical part introduces the fundamental principles of concurrent programming, with an emphasis on its application to parallel task execution, the architecture of the Go programming language, and its built-in mechanisms supporting concurrent programming. The thesis also presents the design of algorithms utilizing these mechanisms together with proofs of their correctness. The practical part is devoted to the design and implementation of a prototype chat server that demonstrates the use of Go's concurrency mechanisms and the proposed algorithms in a real-world application.

Kľúčové slová: Go, goroutine, komunikačný kanál, súbežné programovanie, paralelné programovanie

Keywords: Go, goroutine, channel, concurrency, parallel programming

Děkuji, děkuji, děkuji.

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Úvod	7
2	Communicating Sequential Processes	7
2.1	Komunikácia v CSP	8
2.2	Výrobca a spotrebiteľ v CSP	10
3	Jazyk Go	11
4	Goroutine	12
5	Komunikačné kanály	13
5.1	Implementácia	15
6	Select	17
6.1	Implementácia	18
7	Synchronizačné primitíva	18
7.1	WaitGroup	19
7.2	Mutex a RWMutex	19
8	Chat server	21
8.1	Klientské spojenie	21
8.2	Posielanie správ	22
8.3	Získanie správ	23
8.4	Príkaz select	24
9	Záver	26
10	Conclusion	27
	Literatúra	28

Zoznam tabuliek

1 Úvod

Paralelné programovanie predstavuje dôležitú súčasť moderného vývoja softvéru. S rastúcou zložitou softvérových systémov a rozšírením viacjadrových procesorov rastie požiadavka na efektívne využitie výpočtových prostriedkov prostredníctvom paralelného spracovania úloh. Preto pribúdajú riešenia a mechanizmy umožňujúce spracovanie viacerých úloh súčasne. Tradičný prístup založený na zdieľanej pamäti však prináša komplikácie, ako sú chyby súbehu alebo uviaznutie. Z tohto dôvodu vznikajú prístupy založené na synchronizácii procesov prostredníctvom komunikácie a výmeny správ medzi procesmi. Jedným z významných konceptov v oblasti súbežného programovania je model Communicating Sequential Processes (CSP) [1]. Programovací jazyk Go implementuje princípy modelu CSP prostredníctvom goroutín a komunikačných kanálov. Cieľom bakalárskej práce je predstaviť princípy súbežného programovania v jazyku Go, vysvetliť ich teoretické základy vychádzajúce z modelu CSP a demonštrovať ich využitie pri návrhu a implementácii prototypu chat servera. Práca sa venuje teoretickému opisu mechanizmov súbežného programovania, návrhu paralelných algoritmov pre server a analýze ich správnosti prostredníctvom dôkazov.

2 Communicating Sequential Processes

Communicating Sequential Process (CSP) je formálny model súbežného programovania, ktorý predstavil Charles Antony Richard Hoare v roku 1978.[2] Tento prístup definuje spôsob, akým medzi sebou súbežne vykonávané procesy komunikujú a koordinujú svoje vykonávanie. Na rozdiel od tradičných prístupov založených na zdieľanej pamäti, CSP je založené na komunikácii medzi procesmi, ktorá predstavuje hlavný mechanizmus ich koordinácie a synchronizácie. [1] Koordinácia určuje spôsob spolupráce procesov pri riešení spoločnej úlohy, zatiaľ čo synchronizácia zabezpečuje správne časové usporiadanie ich vykonávania. Procesy si neposkytujú priamy prístup k pamäti, čím sa znižuje riziko vzniku chýb súbehu. Základnou myšlienkou CSP je, že program sa skladá z viacerých sekvenciálnych procesov vykonávaných súbežne, ktoré medzi sebou komunikujú prostredníctvom vstupno-výstupných operácií. [1]

Model CSP je založený na troch základných princípoch, ktoré umožňujú vytvárať bezpečné súbežné programy:

- strážené príkazy,
- paralelná kompozícia procesov,
- komunikácia pomocou vstupu a výstupu.

Koncept strážených príkazov umožňuje riadiť vetvenie a nedeterministické správanie. Vetvenie znamená výber jednej z viacerých možných vetiev programu podľa splnenia podmienky. Nedeterministické správanie nastáva v situácii, keď

je súčasne splnených viac podmienok a model nepredpisuje, ktorá z možných vetiev bude vykonaná. Dôležité je iba to, že bude zvolená jedna z povolených možností. [1] Ide o príkazy, ktorých vykonanie je podmienené splnením logickej podmienky. Ak podmienka nie je splnená, proces čaká, kým je podmienka splnená. Paralelná kompozícia umožňuje vytvoriť program pozostávajúci z viacerých procesov, ktoré sa vykonávajú súčasne, nezávisle od seba a končia až po ukončení všetkých procesov. Komunikácia pomocou vstupu a výstupu znamená, že procesy spolu komunikujú výhradne pomocou vstupno-výstupných operácií. Výstupná operácia predstavuje odoslanie hodnoty procesom, zatiaľ čo vstupná operácia predstavuje prijatie hodnoty iným procesom. Komunikácia sa uskutoční iba vtedy, keď sú odosielateľ aj prijímateľ pripravení vykonať príslušné operácie. Každá komunikácia predstavuje synchronizačný bod medzi dvoma procesmi, pretože prenos údajov prebieha iba vtedy, keď sú odosielateľ aj prijímateľ pripravení komunikovať. Procesy tak nezdieľajú pamäť a ich komunikácia prebieha synchronizovane. [1]

V CSP možno chápať každý proces ako samostatnú výpočtovú jednotku s vlastným stavom, ktorá vykonáva príkazy v definovanom poradí. Na opis správania procesu sa používa pojem *udalosť*. Udalosť predstavuje pozorovateľnú akciu procesu. Nie je dôležitá implementácia, ale navonok pozorovateľné správanie. Udalosť predstavuje okamih pozorovateľný ostatnými procesmi. Postupnosť udalostí sa označuje ako *stopa*. Stopa predstavuje chronologický záznam všetkých udalostí, ktoré proces od svojho vzniku vykonal. [1] Správanie procesu je možné opísať pomocou postupnosti udalostí, ktoré proces počas svojho vykonávania vyvolá, bez potreby poznať jeho implementáciu. Tento spôsob opisu je jeden zo základných princípov formálneho modelovania v CSP. Procesy nie sú izolované. [1] Ich spolupráca je kľúčová na dosiahnutie požadovaného výsledku a je realizovaná pomocou synchronizovanej komunikácie. Všetky informácie získava alebo odosiela prostredníctvom výmeny správ. Tento prístup ale môže viesť k situácii, kedy procesy čakajú bez možnosti pokračovať, teda uviaznutiu pre jeho správanie, kedy odosielateľ čaká na príjemcu a naopak.

2.1 Komunikácia v CSP

Hlavnou vlastnosťou CSP je prenos dát medzi procesmi prostredníctvom zasielania správ. Hodnota odoslaná jedným procesom je priamo priradená premennej v druhom procese. Procesy musia byť pripravené na komunikáciu v rovnakom okamihu. Proces nemôže pokračovať, kým druhý proces nie je pripravený na komunikáciu. Prenos hodnoty je atomický a synchronizovaný, teda nemôže prísť k čiastočnému prenosu alebo nekonzistentnému stavu. [2] Komunikácia medzi procesmi súčasne zabezpečuje prenos dát aj synchronizáciu procesov. Tento spôsob komunikácie zjednodušuje uvažovanie o správnosti programu, pretože eliminuje potrebu zamykania a konflikty pri prístupe k pamäti. [1]

Na opis správania procesov CSP používa vlastnú syntax. Jednotlivé symboly predstavujú základné operácie komunikácie, synchronizácie a riadenia toku prog-

ramu. [1]

Syntax	Význam
$a := b$	Priradenie hodnoty premennej.
$P?x$	Prijatie hodnoty od procesu P a uloženie do premennej x .
$P!x$	Odoslanie hodnoty x procesu P .
$SKIP$	Prázdny príkaz, ktorý nevykonáva žiadnu operáciu a okamžite končí.
$A \rightarrow B$	Po vykonaní udalosti A pokračuje proces vykonávaním procesu B . Tento zápis predstavuje prefixový operátor v CSP.
$G \rightarrow S$	Stráženy príkaz (guard). Ak je splnená podmienka G , vykoná sa príkaz S . V opačnom prípade proces čaká, kým sa podmienka nestane pravdivou.
$[]$	Alternatívny príkaz. Vyberie jednu zo strážených alternatív, ktorých podmienka je splnená. Ak je splnených viac podmienok, výber je nedeterministický.
$*[]$	Repetitívny príkaz. Opakovane vykonáva alternatívny príkaz, pokiaľ je splnená aspoň jedna podmienka.
$(i : 1..n)$	Rozsah indexov používaný pri komunikácii s ľubovoľným procesom zo skupiny procesov.
$P Q$	Paralelné vykonávanie procesov P a Q .
μX	Rekurzívna definícia procesu umožňuje definovať proces odkazujúci na svoju vlastnú definíciu, čím možno modelovať opakujúce sa alebo nekonečné správanie.
$\dots$	Ľubovoľná činnosť vykonaná procesom po úspešnej komunikácii, ktorej implementácia nie je z pohľadu modelu CSP podstatná.

Priradenie hodnoty premennej má rovnaký význam ako v bežných imperatívnych programovacích jazykoch. Vstup a výstup sú synchronizované tak, že proces posielajúci výstupnú operáciu musí čakať, kým iný proces nevykoná zodpovedajúcu vstupnú operáciu. Napríklad, ak proces P odošle hodnotu operáciou $P!x$, druhý proces musí byť pripravený prijať túto hodnotu operáciou $P?y$. Až potom dôjde k samotnému prenosu údajov. Obidve operácie tvoria jednu spoločnú komunikačnú udalosť. [1]

Štruktúrované príkazy predstavujú zloženejšie riadiace konštrukcie vytvorené z jednoduchých príkazov.

Alternatívny príkaz slúži na výber jednej z viacerých možných alternatív podľa splnenia strážených podmienok.

```
[
  x > 0 -> P!x
]
```

$$y > 0 \rightarrow Q!y$$

$$]$$

Ak je splnená iba podmienka $x > 0$, vykoná sa prvá alternatíva. Ak je splnená iba podmienka $y > 0$, vykoná sa druhá alternatíva. Ak sú splnené obe podmienky, model CSP nedeterminizmom vyberie jednu z nich. Ak nie je splnená ani jedna podmienka, proces čaká, kým sa niektorá z podmienok nestane pravdivou. [1] Alternatívny príkaz slúži na modelovanie situácií, kde správanie programu závisí od aktuálnych podmienok alebo interakcie s inými procesmi.

Repetitívny príkaz predstavuje cyklus alternatívnych príkazov.

$$* [$$

$$\quad in?x \rightarrow out!x$$

$$[]$$

$$\quad stop \rightarrow SKIP$$

$$]$$

Proces opakovane prijíma hodnoty zo vstupu *in* a odosiela ich na výstup *out*. Cyklus pokračuje dovtedy, kým je možné vykonať aspoň jednu z alternatív. Ak už nie je splnená žiadna podmienka, vykonávanie sa ukončí. [1] Repetitívny príkaz sa používa na modelovanie situácií, kde program opakovane reaguje na prichádzajúce správy alebo udalosti.

2.2 Výrobca a spotrebiteľ v CSP

Pri riešení zložitejších úloh často komunikuje viacero procesov s jedným spoločným procesom, ktorý koordinuje ich činnosť. Takýto proces možno chápať ako monitor. Monitor ako synchronizačný objekt prijímajúci požiadavky od ostatných procesov, zabezpečujúci ich vzájomnú synchronizáciu a rozhodujúci o poradí ich spracovania. V modeli CSP monitor nepredstavuje zdieľanú pamäť, ale samostatný proces komunikujúci prostredníctvom výmeny správ. [1] Procesy musia byť odlišne pomenované napr. „Výrobca“ a „Spotrebiteľ“, alebo mať odlišný index napríklad „X(0)“. Každá komunikácia musí mať jednoznačne určený svoj zdroj alebo cieľ. Keď je monitor pripravený komunikovať s ktorýmkoľvek používateľským procesom, zväčša ktorý príde prvý, používa strážený príkaz s rozsahom. Predpokladajme, že existuje sto procesov označených ako X(1) až X(100), ktoré komunikujú s jedným monitorom. Monitor nevie dopredu, ktorý proces odošle požiadavku ako prvý. Preto musí byť pripravený prijať správu od ktoréhokoľvek z nich.

$$[(i : 1..100); X(i)?(value\ parameters) \rightarrow \dots; X(i)!(results);]$$

Premenná „*i*“ identifikuje proces, ktorý komunikáciu inicioval. Monitor ju následne využije pri odoslaní výsledku späť správnenému procesu.

Ak monitor nie je pripravený na komunikáciu od konkrétneho procesu napríklad „X(*j*)“, môže byť vstupný príkaz doplnený o booleovskú podmienku, takzvanú stráž. Napríklad možno zabrániť tomu, aby monitor dvakrát po sebe obslúžil

požiadavku od toho istého procesu. To možno dosiahnuť podmienkou $i \neq j$, kde premenná j uchováva identifikátor naposledy obsluženého procesu. Predstavme si, že monitor obsluhuje požiadavky klientov. Ak by jeden klient posielal požiadavky nepretržite, ostatné procesy by mohli zostať dlhý čas neobslužené. Preto možno pomocou stráženej podmienky dočasne zablokovat opätovné prijatie požiadavky od toho istého procesu, kým nebude spracovaná požiadavka od iného procesu.

$$*[(i : 1..100)i \neq j; X(i)?(parameters) \rightarrow \dots; i := j;]$$

V tomto prípade monitor neprijme ďalšiu požiadavku od procesu „X(j)“, kým nebude spracovaná požiadavka iného procesu. Rovnakým spôsobom možno pomocou strážených podmienok odložiť spracovanie požiadaviek, ktoré by v danom okamihu porušili pravidlá plánovania alebo iné obmedzenia systému. [2]

Ako príklad možno uviesť problém výrobcu a spotrebiteľa. Predpokladajme, že existuje viacero procesov „Výrobca“, ktoré vytvárajú údaje, a viacero procesov „Spotrebiteľ“, ktoré ich spracúvajú. Medzi nimi sa nachádza monitor predstavujúci spoločný zásobník. Výrobcovia odosielaajú vytvorené údaje monitoru, ktorý ich uloží, zatiaľ čo spotrebiteľia od monitora požadujú ďalšiu dostupnú položku. Monitor zabezpečuje, aby sa údaje odovzdávali korektne aj v prípade, že viac výrobcov alebo spotrebiteľov komunikuje súčasne.

$$\begin{aligned} \text{Výrobca}(i) &= \text{Monitor!item} \rightarrow \text{Výrobca}(i) \\ \text{Spotrebiteľ}(j) &= \text{Monitor?item} \rightarrow \text{Spotrebiteľ}(j) \\ \text{Monitor} &= *[(i : 1..m); (j : 1..n); \\ &\quad \text{Výrobca}(i)?\text{item} \rightarrow \text{Spotrebiteľ}(j)!\text{item}] \end{aligned}$$

V uvedenom príklade môže existovať ľubovoľný počet výrobcov a spotrebiteľov. Indexy i a j identifikujú konkrétny proces, ktorý práve komunikuje s monitorom. Monitor prijíma údaje od ľubovoľného výrobcu a poskytuje ich ľubovoľnému spotrebiteľovi, pričom zabezpečuje správnu synchronizáciu komunikácie medzi všetkými procesmi.

3 Jazyk Go

Programovací jazyk Go bol vyvinutý spoločnosťou Google s cieľom vytvoriť jednoduchý, efektívny a výkonný jazyk. Jazyk je vhodný na vývoj moderných aplikácií. Pri návrhu bol kladený dôraz na rýchlosť kompilácie, jednoduchú syntax, bezpečnosť práce s pamäťou a natívnu podporu súbežného programovania. Go patrí medzi kompilované, staticky typované programovacie jazyky a poskytuje automatickú správu pamäte pomocou garbage collectoru. [3] Go obsahuje vstavané mechanizmy na paralelné a súbežné programovanie, ktorým sa venuje táto práca. V Go je súbežnosť súčasťou návrhu jazyka a predstavuje jednu z jeho najvýznamnejších vlastností.

Go je staticky typovaný jazyk, čo znamená, že typ každej premennej je známy už počas kompilácie. Vďaka tomu je možné odhaliť chyby s tým spojené ešte pred spustením programu. Typ premennej môže byť uvedený explicitne:

```
var <variable_name> <type> = <value>
```

alebo pomocou skrátenej deklarácie, ktorú vie kompilátor odvodiť:

```
<variable_name> := <value>
```

Skrátená deklarácia patrí medzi najčastejšie používané zápisy v Go a znižuje množstvo potrebného kódu. [3]

Go podporuje dátové typy:

celočísenné typy	int, int64, uint
reálne čísla	float32, float64
logický typ	bool
reťazce	string
polia	array
rezy	slice
mapy	map
štruktúry	struct

Funkcie sa deklarujú pomocou kľúčového slova „func“. Návratový typ sa zapisuje za zoznam argumentov. [3]

```
func <name>(<arguments>, <argument>) <return_type> {  
    return <value>  
}
```

Významnou vlastnosťou Go je možnosť viacerých návratových hodnôt z funkcie. Často sa využíva pri návrate výsledku spolu s informáciou o chybe:

```
value, err := ReadFile(path)
```

Takýto spôsob spracovania chýb znižuje potrebu používania výnimiek a predstavuje štandardný mechanizmus ošetrovania chýb v Go. [3]

Jedným z hlavných cieľov Go je jednoduchosť. Jazyk preto neobsahuje preťažovanie funkcií, preťažovanie operátorov, implicitné pretypovanie dátových typov ani aritmetiku ukazovateľov. Natívne podporuje mechanizmy súbežného a paralelného programovania. Poskytuje primitíva ako goroutiny, komunikačné kanály, synchronizačné primitíva v balíku sync a príkaz select. [4]

4 Goroutine

Goroutiny (*goroutines*) je funkcia, ktorá sa vykonáva súbežne. Nie sú to vlákna operačného systému ani vlákna riadené runtime prostredím. Ide o vyššiu abstrakciu nazývanú v jazyku Go goroutiny. Goroutiny sú súbežné podprogramy, ktoré

nemôžu byť prerušené zvonka. Obsahujú však body, v ktorých môžu byť pozastavené a znovu spustené. Na rozdiel od vlákien je prepínanie medzi goroutinami rýchlejšie približne o 1,242 μ s.[5] Goroutiny sa dajú spustiť pomocou kľúčového slova *go*.

```
func main{
    go SayHello()
}

func SayHello(){
    fmt.Println("Hello")
}
```

Goroutiny možno implementovať ako anonymné funkcie.

```
go func(){
    fmt.Println("Hello")
}
```

Pri použití anonymnej funkcie je potrebné funkciu bezprostredne zavolať. Vieme goroutinu priradiť do premennej.

```
sayHello := func(){
    fmt.Println("Hello")
}

go sayHello()
```

Goroutiny sú špecifické pre jazyk Go, pretože sú integrované do runtime prostredia Go. Goroutiny si sami neurčujú body pozastavenia. Runtime Go sleduje ich správanie a automaticky ich pozastaví a znovu spustí, keď sú opätovne pripravené pokračovať. Súbežné vykonávanie gorutín zabezpečuje plánovač runtime prostredia Go. [3]

Go používa plánovač „M:N“ a model súbežnosti fork-join.[5] Teda na „M“ vláknach sa vykonáva „N“ gorutín. Goroutiny sú mapované na tieto vlákna a go runtime sa stará o ich rozdelenie a efektívne vykonávanie. Stará sa o ich spúšťanie, ak je nejaká goroutina blokováaná, runtime môže spustiť inú.

Podľa modelu fork-join, „Fork“ znamená, že program môže vytvoriť novú goroutinu, ktorá sa vykonáva súbežne s rodičovským kódom. Naopak, „Join“ označuje, že sa tieto vetvy spoja v mieste nazývanom join point. Samotné vytvorenie gorutiny nezabezpečuje čakanie na jej dokončenie.

5 Komunikačné kanály

Komunikačné kanály predstavujú jeden zo základných mechanizmov synchronizácie a komunikácie medzi goroutinami. Sú zabudované referenčným typom,

ktorý umožňuje bezpečný prenos dát. Ich návrh vychádza z Communicating Sequential Processes, ktorý zdôrazňuje komunikáciu medzi procesmi miesto zdieľania pamäte.[5]

Komunikačné kanály sa deklarujú pomocou kľúčového slova *chan* a vždy sa deklarujú aj s určitým typom. To znamená, že kanál môže prenášať hodnoty iba tohto typu. Komunikačné kanály patria k referenčným typom, preto ich deklarácia nevytvára potrebnú pamäťovú štruktúru. Pamäť sa alokuje pomocou zabudovanej funkcie *make*.

```
make(chan type , buffer_size )
```

Takto vytvorený komunikačný kanál umožňuje zapisovanie a čítanie hodnôt. Zápis do komunikačného kanála umožňuje operátor *<-*, ktorý sa umiestňuje na pravú stranu názvu komunikačného kanála, pre čítanie sa operátor umiestňuje na ľavú stranu názvu komunikačného kanála.

```
channel <- data  
variable = <- channel
```

Dôležitou vlastnosťou komunikačného kanála je jeho blokujúce správanie. Ak goroutina zapisuje do komunikačného kanála, ktorý je plný, je jej proces blokovaný kým iná goroutina z komunikačného kanála hodnotu neprečíta. Podobne čítanie z prázdneho komunikačného kanála je blokované kým nie je do komunikačného kanála zapísaná nová hodnota.

Komunikačný kanál je možné zavrieť pomocou funkcie *close*. Pri čítaní z kanála je možné získať druhú návratovú hodnotu typu *bool*, ktorá indikuje jeho otvorenie. Uzavretie teda signalizuje prijímateľovi, že do komunikačného kanála nebude viac zapisované. Ak odosielateľ pošle dáta do uzavretého komunikačného kanálu, program vráti error *panic: send on closed channel*. Po uzavretí komunikačného kanálu, prijímateľ nečaká na hodnotu ale obdrží prázdnu hodnotu a hodnotu „False“ z druhej návratovej hodnoty. Nasledujúci príklad demonštruje použitie.

```
1      var channel = make(chan int , 1)  
2  
3      go func () {  
4          channel <- 42  
5      }()  
6  
7      result := <- channel  
8      close(channel)
```

Na riadku jedna si vytvoríme komunikačný kanál s referenčným typom *int* a veľkosťou buffera jedna. Na riadku tri spustíme novú goroutinu. Na riadku štyri v goroutine uložíme do komunikačného kanála hodnotu 42. Na riadku sedem hlavné vlákno čaká na dáta z komunikačného kanála. Ak dáta prídu, uloží do premennej „result“ dáta z komunikačného kanála, v tomto prípade 42. Na riadku osem uzavrieme komunikačný kanál.

Každý komunikačný kanál môže byť bufferovaný alebo nebufferovaný. Nebufferovaný komunikačný kanál nemá žiadnu kapacitu a každá operácia zápisu musí byť okamžite spravovaná operáciou čítania.

Bufferovaný komunikačný kanál sa definuje pomocou voliteľného druhého argumentu funkcie `make`. Bufferovaný komunikačný kanál obsahuje buffer s definovanou kapacitou, ktorá umožní uložiť určitý počet hodnôt do bufferu bez okamžitého čítania. Bufferovaný komunikačný kanál zablokuje zápis až v momente keď je buffer plný.

Jazyk Go umožňuje definovanie funkcií, kde ako parameter definujeme komunikačný kanál. Pri týchto funkciách je umožnené definovanie jednosmerných kanálov, ktoré podporujú iba zápis alebo čítanie. Tieto komunikačné kanály sa používajú ako parametre funkcií, aby určili akým spôsobom sa komunikačný kanál v danej časti programu používa. Tento mechanizmus zvyšuje bezpečnosť programu, pretože kompilátor dokáže zachytiť nesprávne použitie.

5.1 Implementácia

V runtime prostredí je komunikačný kanál implementovaný ako štruktúra `hchan`. Obsahuje počet hodnôt v kanáli, maximálnu kapacitu, ukazovateľ na buffer, fronty čakajúcich goroutín a synchronizačný zámok. Zjednodušene štruktúra obsahuje:

<code>qcount uint</code>	Počet prvkov v bufferi
<code>dataqsiz uint</code>	Veľkosť jednej hodnoty na bufferi
<code>buf unsafe.Pointer</code>	Pointer na buffer
<code>sendx uint</code>	Index pre zápis
<code>recvx uint</code>	Index pre čítanie
<code>recvq waitq</code>	Čakajúci prijímatelia
<code>sendq waitq</code>	Čakajúci odosielatelia
<code>closed uint32</code>	Hodnota určujúca či je komunikačný kanál otvorený alebo zatvorený
<code>lock mutex</code>	Mutex na prístup k bufferu

Komunikačný kanál funguje ako kruhový buffer, kde dva indexy označujúce aktuálnu pozíciu a pozíciu posledného pridaného prvku. Pri dosiahnutí maximálneho indexu, ktorý určuje možný počet prvkov v bufferi, sa pokračuje indexom nula. Ak sú indexy na rovnakej pozícii, značí to prázdny buffer. Ak je index posledného pridaného prvku o jedna menší ako index aktuálnej pozície alebo index posledného pridaného prvku je maximálny a index aktuálnej pozície je nula, značí to plný buffer. Komunikačný kanál je doplnený o mechanizmy blokujúce goroutiny.[6] Implementácia front „sendq“ a „recvq“ je hlavný prvok na synchronizáciu goroutín, kedy odosielateľ čaká na príjemcu a príjemca čaká na odosielateľa. To zabezpečuje bezpečnú synchronizáciu a absenciu problému súbehu zámok v aplikačnom kóde. Zámok „mutex“ využíva komunikačný kanál pri operáciách, ktoré môžu meniť jeho stav. Teda pri posielaní alebo odosielaní správ sa komunikačný kanál uzamkne aby nenastal nekonzistentný stav komunikačného kanála.

Komunikačný kanál má operácie prijímanie a odosielanie hodnoty. Komunikačný kanál odosiela a prijíma dve hodnoty. Posielanú hodnotu a hodnotu typu `bool`, ktorá značí, či je kanál otvorený. Odosielanie hodnoty je implementované funkciou *chansend*. Pri tejto operácii je komunikačný kanál uzamknutý, preto túto operáciu možno vykonať iba jedna goroutina v určitom čase. Jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene, rozlišuje tri stavy. Ak existuje čakajúci príjemca, teda je hodnota v `recvq` goroutine, hodnota sa prenesie priamo bez použitia bufferu.

```
if receiver exist:
    copy data directly to receiver
    wake up receiver
```

Najrýchlejší scenár, nakoľko sa neukladá hodnota do bufferu. Ďalší stav, ak neexistuje čakajúci príjemca ale existuje voľné miesto v bufferi. Tak sa hodnota uloží na buffer a zvýši sa počítadlo.

```
if buffer not full:
    store value in buffer
    increment qcount
```

Tretí stav ak neexistuje čakajúci príjemca a neexistuje voľné miesto na bufferi. V takom prípade sa goroutina uloží do fronty „`sendq`“ a čaká na uvoľnenie miesta na bufferi.

Prijímanie hodnoty je implementované funkciou *chanrecv*. Rovnako jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene rozlišuje tri stavy. Ak existuje čakajúci odosielateľ, teda je hodnota v `sendq`, hodnota sa prenesie priamo bez použitia bufferu.

```
if sender exist:
    receive value directly
    wake sender
```

Ak neexistuje čakajúci odosielateľ ale buffer obsahuje aspoň jednu hodnotu, je táto hodnota načítaná a zníži sa hodnota počítadla.

```
if buffer not empty:
    read value from buffer
    decrement qcount
```

Ak neexistuje čakajúci odosielateľ a buffer neobsahuje žiadnu hodnotu. V takom prípade sa goroutina uloží do fronty „`recvq`“ a čaká na odosielateľa.

Operácia uzavretia komunikačného kanála nastaví „`closed`“ na hodnotu „`False`“. Zároveň sa odblokujú všetky čakajúce goroutiny v „`recvq`“. Goroutiny v prvej návratovej hodnote obdržia prázdnu hodnotu, čo nie je vždy chcené a je potrebná reakcia programu na druhú návratovú hodnotu. Pri odosielaní na uzavretý komunikačný kanál, program vráti chybu.

6 Select

Príkaz `select` slúži na koordináciu komunikácie medzi goroutinami. Zatiaľ čo komunikačný kanál slúži na synchronizáciu a prenos dát, príkaz `select` umožňuje efektívne kombinovať a riadiť komunikačné kanály na základe aktuálneho stavu programu. Používa sa na spájanie viacerých komunikačných kanálov, synchronizáciu, implementáciu časových limitov alebo reakciu na viacero udalostí súčasne. `Select` sa používa v spojení s komunikačnými kanálmi a má podobnú syntax ako príkaz `switch`. Pozostáva z príkazu *select* a viacerých vetiev *case*, kde každá vetva reprezentuje operáciu nad kanálom. Môže to byť operácia čítania alebo zápisu. Jeho hlavnou vlastnosťou pre synchronizáciu je jeho blokovacie správanie. Proces je blokový kým nie je pripravená na vykonanie aspoň jedna z jeho vetiev. Pripravenosť znamená, že pri zápise do komunikačného kanála, nie je komunikačný kanál zaplnený a pri čítaní z komunikačného kanála, komunikačný kanál obsahuje dáta alebo je uzavretý. Pre neblokujúce správanie príkazu `select` slúži jeho posledná vetva *default*, ktorá sa vykoná v prípade, ak nie je pripravená na vykonanie žiadna z ostatných vetiev. Rozdiel oproti príkazu `switch` je, že jednotlivé vetvy sa nevyhodnocujú postupne ale všetky súčasne. Ak je pripravená niektorá z vetiev, tak sa vykoná. Ak je pripravených viacero vetiev v rovnakom okamihu, runtime Go vyberie jednu z pripravených a vykoná ju. Výber je rovnomerne rozdelený medzi všetky pripravené vetvy, preto má každá rovnakú pravdepodobnosť vykonania[5]. Príkaz `select` umožňuje implementáciu časových limitov. Pomocou funkcie „`time.After`“ z balíka „`time`“, ktorá vracia komunikačný kanál aktivovaný po uplynutí zadaného času. Tento prístup umožňuje riešenie situácií, kde nie je žiaduce čakať neobmedzene dlho.

```
1      ch := make(chan int , 1)
2      go func () {
3          for {
4              i := rand.Intn(5)
5              ch <- i
6              time.Sleep(time.Duration(i) * time.Second)
7          }
8      }()
9
10     for {
11         select {
12             case num := <-ch:
13                 fmt.Println(num)
14
15             case <-time.After(3 * time.Second):
16                 return
17         }
18     }
```

V príklade sme si vytvorili komunikačný kanál a jednoduchú funkciu, ktorá vy-

berie náhodné číslo od nula do štyri, pošle ho cez komunikačný kanál a počká taký počet sekúnd. V nekonečnom cykle máme príkaz `select`, ktorý je blokovaný kým nie je cez komunikačný kanál poslaná hodnota alebo kým vyprší interval troch sekúnd, kedy sa ukončí. S každou interakciou sa vytvorí nový časovač. Na tom môžeme vidieť, že `select` si vyhodnocuje všetky vetvy súčasne. Ak je aspoň jedna pripravená na vykonanie, vykoná ju, inak je blokovaný.

6.1 Implementácia

V runtime Go je vetva „case“ v príkaze `select` reprezentovaná štruktúrou `scase`, ktorá obsahuje ukazovateľ na komunikačný kanál a nepovinný ukazovateľ na prenášané dáta.

```
type scase struct {  
    c *hchan  
    elem unsafe.Pointer  
}
```

[7] Vetvy sú následne uložené do poľa, kde sa periodicky testuje prvok na náhodnej pozícii. Používa sa na to funkcia „`cheaprandn`“. Použitím náhodného testovania vetiev sa zabezpečí rovnaká pravdepodobnosť vykonania pre každú vetvu. Pred vykonaním výberu sa ale uzamkne príslušný komunikačný kanál aby sa predišlo uviaznutiu.

Najprv `select` hľadá operáciu, ktorá je pripravená na vykonanie bez blokovania goroutín s príkazom `select`. Teda snaží sa vykonať príslušnú operáciu na komunikačný kanál bez toho, aby sa zapísala do fronty čakateľov, „`recvq`“ pre operáciu prijatia a „`sendq`“ pre operáciu odoslania. Ak nie je pripravená žiadna vetva na vykonanie tak sa goroutina s príkazom `select` zapíše do každého komunikačného kanála do fronty čakateľov. Následne je blokovaná až kým ju neprebudí jeden z komunikačných kanálov. V takom prípade sa odstráni z fronty čakateľov u ostatných kanálov a pokračuje operáciou výberu. Ak je `select` použitý s vetvou „`default`“ a žiadna operácia nie je pripravená, vykoná sa vetva „`default`“ bez blokovania.

7 Synchronizačné primitíva

Jazyk Go poskytuje `sync` balík, ktorý obsahuje synchronizačné primitíva. Tieto synchronizačné primitíva sú používané pri nízkoúrovňovom prístupe do pamäte. Rozdiel medzi ostatnými jazykmi a jazykom Go je, že Go poskytuje novú sadu synchronizačných primitív pre súbežnosť nad sadou primitív pre synchronizáciu prístupu k pamäti.

7.1 WaitGroup

Je to nástroj z balíka `sync`, ktorého účelom je umožniť programu počkať na dokončenie skupiny goroutín pred tým, než bude pokračovať. Mechanizmus sa využíva v prípade, že sa vykonáva viacero nezávislých úloh. `WaitGroup` si môžeme predstaviť ako bezpečný čítač. Tento čítač eviduje počet goroutín, ktoré ešte nedokončili svoju činnosť. Čítač sa zvyšuje pomocou metódy `Add(n)`, kde „n“ udáva počet goroutín, ktoré sa majú vykonať. Čítač sa znižuje pomocou metódy `Done()`. Program je zablokovaný na metóde `Wait()`, kým hodnota čítača nedosiahne nulovú hodnotu. Nasledujúci príklad demonštruje použitie.

```
1      var wg sync.WaitGroup
2
3          for i := 0; i < 10; i++ {
4              wg.Add(1)
5              go func(i int) {
6                  defer wg.Done()
7                  fmt.Println("Goroutine", i)
8                  time.Sleep(200)
9              }(i)
10         }
11         wg.Wait()
12         fmt.Println("End")
```

Na prvom riadku si definujeme `WaitGroup`. Následne v cykle na riadku tri zvýšime hodnotu `WaitGroup` o jedna. Na riadku štyri pustíme novú goroutinu. V goroutine na riadku päť zabezpečíme, že po dokončení činnosti, zníži hodnotu `WaitGroup` o jedna. Následne vykoná vypís a čakanie pre simuláciu paralelného výpočtu. Na riadku jedenásť program čaká, kým hodnota `WaitGroup` bude nulová. následne vypíše „End“.

7.2 Mutex a RWMutex

Mutex je nástroj z balíka `sync`, ktorého účelom je chrániť časť programu so zdieľanými zdrojmi, ku ktorým môže pristupovať iba jedna goroutina. Pred vstupom do časti programu so zdieľanými zdrojmi, musí goroutina získať zámok pomocou metódy `Lock()`. Po ukončení činnosti v danej časti programu, uvoľní zámok pomocou metódy `Unlock`. Počas držania zámku, nemôže iná goroutina vstúpiť do tejto časti programu. `RWMutex` rozširuje `Mutex` o zámok, ktorý rozlišuje operácie čítania a zápisu. Metódy na získanie zámku `RLock()` / `Lock()`, metódy na uvoľnenie zámku `RUnlock()` / `Unlock()`. `RWMutex` umožňuje získať zámok pre čítanie viacerým goroutinami, ale pre zápis umožňuje získať zámok jednej goroutine. Počas zápisu nie je umožnené získať zámok pre čítanie. Pri zápise čaká kým sú uvoľnené všetky zámky. Nasledujúci príklad demonštruje použitie.

```
1      var wg sync.WaitGroup
2      var rw sync.RWMutex
```

```

3
4     observer := func(i int) {
5         defer wg.Done()
6         rw.RLock()
7         fmt.Println("Reading started", i)
8         time.Sleep(400 * time.Millisecond)
9         fmt.Println("Reading ends", i)
10        rw.RUnlock()
11    }
12
13    writer := func(i int) {
14        defer wg.Done()
15        defer rw.Unlock()
16        rw.Lock()
17        fmt.Println("Writting started", i)
18        time.Sleep(400 * time.Millisecond)
19        fmt.Println("Writting ended", i)
20    }
21
22    for i := 0; i < 100; i++ {
23        wg.Add(1)
24        if i%25 == 0 {
25            go writer(i)
26        } else {
27            go observer(i)
28        }
29    }
30    wg.Wait()
31    fmt.Println("End")

```

Na riadku dva si definujeme RWMutex. Na riadku štyri si definujeme funkciu čitateľa, ktorá sa pokúsi získať zámok pre čítanie vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok pre čítanie. Na riadku trinásť si definujeme funkciu zapisovateľa, ktorá sa pokúsi získať zámok, vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok. Na riadku dvadsaťdva je cyklus, v ktorom voláme funkciu zapisovateľa ak „i“ modulo 25 je nula, inak voláme funkciu čitateľa. Následne počkáme na dokončenie všetkých goroutín a vypíšeme, že sme skončili. Výsledok tohto kódu bude približne.

```

"Writting started 0"
"Writting ended 0"
"Reading started 1"
"Reading started 2"
"Reading started 5"
...
"Reading ended 98"

```

```
"Reading ended 97"  
"Reading ended 99"  
"Writting started 50"  
"Writting ended 50"  
"Writting started 75"  
"Writting ended 75"
```

Z tohto výsledku vidíme, že RWMutex uprednostňuje čitateľov. Vynútenie zámku pre zapisovateľa je možné pomocou ďalších synchronizačných nástrojov, ako napríklad pomocou komunikačných kanálov.

8 Chat server

Navrhnutý prototyp serveru je založený na architektúre klient-server, kde server zabezpečuje výmenu správ medzi pripojenými klientmi. Komunikácia medzi klientom a serverom je realizovaná cez Transmission Control Protocol (TCP), ktorý zabezpečuje spoľahlivý prenos dát, zachovanie poradia správ a detekciu chýb pri prenose. Server je implementovaný v programovacom jazyku Go, využívajúc jeho vstavanú podporu pre súbežné programovanie prostredníctvom goroutín. Hlavná úloha servera spočíva v prijímaní nových klientskych spojení, spracovaní ich požiadavkov a správe komunikačných miestností.

8.1 Klientské spojenie

```
for {  
    conn, err := ln.Accept()  
    go clientHandle.HandleConnection(conn)  
}
```

Pri spustení server vytvorí TCP listener „ln“. Následne v nekonečnom cykle čaká na nové pripojenie, ak je pripojenie úspešné, vytvorí pre to spojenie novú goroutinu. Tým sa zabezpečí, že každý klient je obslužený súbežne a bez blokovania prijímania ďalších klientov.

Každý klient je obslužený na samostatnej goroutine, ktorá vykonáva funkciu *HandleConnection*. Toto riešenie predstavuje z pohľadu architektúry model:

- jeden hlavný serverový proces
- jedna goroutina na klienta

Riešenie je efektívne, pretože goroutiny sú ľahšie ako klasické vlákna a umožňujú vysokú mieru škálovateľnosti.

Po pripojení je od klienta vyžiadané meno. Následne v nekonečnom cykle prvá podmienka pripojí klienta do komunikačnej miestnosti a v následných cykloch spracúva požiadavky:

- `/fetch` Získanie nových správ v komunikačnej miestnosti
- `/fetchAll` Získanie správ zo všetkých navštívených komunikačných miestností
- `/broadcast` Poslanie správy do všetkých navštívených komunikačných miestností
- `exit` Opustenie komunikačnej miestnosti
- `logout` Odhlásenie sa zo serveru. Teda ukončenie nekonečného cyklu.
- `default` Poslanie správy

8.2 Posielanie správ

Problémom pri posielaní správ je, aby sa jednotlivé správy uložili do databázy správne, a aby nevznikol problém súbehu. Problém sme riešili pomocou komunikácie medzi procesmi s použitím komunikačného kanála. Využili sme jeho blokujúce správanie a prenos dát medzi procesmi. Na správu správ sme použili štruktúru *Message*, kde sa eviduje odosielateľ a obsah správy. Pre každú komunikačnú miestnosť sme použili štruktúru *ChatLog*, ktorá obsahuje *Message* v zozname ako poslané správy v danej komunikačnej miestnosti. Táto štruktúra obsahuje aj komunikačný kanál, pomocou ktorého sa správy ukladajú súbežne.

```

1      for msg:= range cl.writeCh {
2          cl.Message = append( cl.Message , msg)
3      }
```

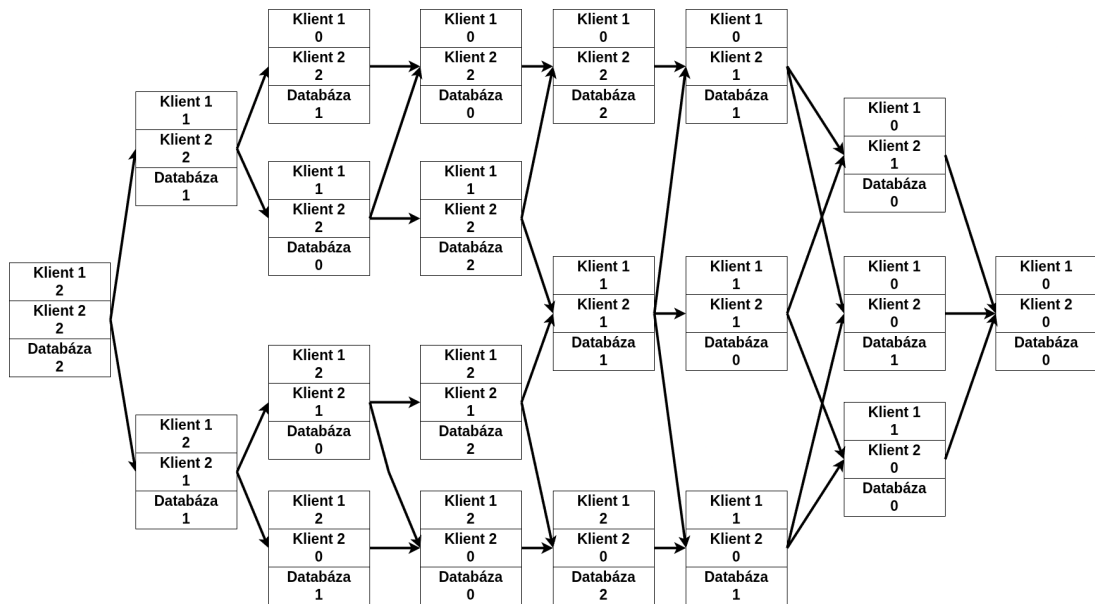
Týmto je zabezpečené, že komunikačný kanál čaká kým doň nie je zapisované. Ak chce klient zapísať správu, tak komunikačným kanálom pošle správu, ktorá sa pridá do zoznamu.

```
cl.writeCh <- message.Message{User: user , Message: text }
```

Klient posiela správu cez kanál. Aj keď môže viacero klientov poslať správu súčasne, komunikačný kanál umožňuje poslanie iba jednej správy, ktorá sa spracuje predtým ako je umožnené ďalšiemu klientovi poslanie správy cez komunikačný kanál. Je preto klient pozastavený a čaká kým je opäť program pripravený spracovať ďalšiu správu. Zapísané v pseudokóde:

go handler() go channel()	
ch <- message return	for { message := <- ch save(message) }

Vytvoríme groutine pre ukladanie dát, ktorá ukladá správy do databázy. Ďalej vytvoríme goroutinu predstavujúcu obsluhu klienta. Vidíme, že aj pri viacerých klientoch nevznikne problém súbehu a dáta sa zapíšu do databázy správne. Využívame blokovacie správanie komunikačného kanála, kedy pri prijatí viacerých správ odosielateľ čaká, kým je prijímateľ pripravený spracovať dáta. Dokážeme si to na príklade dvoch procesov klientov a procesu databázy.



Tu vidíme, že procesy klientov sú blokované, kým proces ukladania dát je pripravený prijať dáta v stave Databáza dva. Poslanie dát prebieha synchronne v stave Databáza dva a Klient dva. Inak sa procesy vykonávajú nezávisle. Preto vždy dôjdeme k požadovanému výsledku.

8.3 Získanie správ

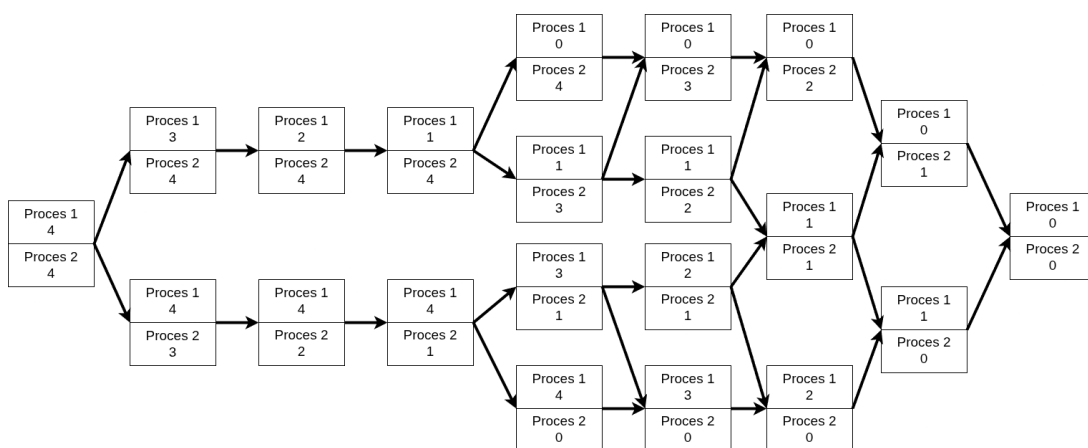
Problém pri získaní správ je aby sa získali všetky správy neporušené a odoslali sa klientovi. Na tento problém sme použili mutex a WaitGroup.

```

1      var chatlog chatLog.ChatLog
2      for _, room := range rooms {
3          wg.Add(1)
4          go func(room string, chatLog *chatLog.ChatLog) {
5              defer wg.Done()
6              mutex.Lock()
7              getMessages()
8              mutex.Unlock()
9              mapMessages()
10         }(room, chatLog)
11     }

```

Pre každú navštívenú komunikačnú miestnosť spustíme goroutinu a zvýšime WaitGroup. Každá goroutina si potom zoberie zámok a získa správy. Následne uvoľní zámok a namapuje správy do odpovede klientovi. Hlavný proces čaká, kým všetky goroutiny dokončia svoju činnosť. Správy sa získavajú cez komunikačný kanál. Ten definujeme ako odkaz na štruktúru. Preto je mutex potrebný, aby nenastalo, že sa pred získaním správ prepíše odkaz na komunikačný kanál a tým sa nezískajú všetky správy, ale nežiadane sa niektoré správy duplikujú.



Vidíme, že procesy nemôžu naraz vstúpiť do stavu tri a dva, nakoľko v stave štyri si berú zámok a stavom dva ho uvoľňujú. Mimo tejto sekcie môžu ľubovoľne pokračovať. Použitím mutexu sme zabezpečili, že do kritickej sekcie sa dostane vždy iba jeden proces v určitom čase. Preto vždy dôjdeme k požadovanému výsledku.

8.4 Príkaz select

Príkaz select sme použili na klientovej strane, kde rieši problém získavania správ. Problém je, aké správy klient chce získať.

```

1 select {
2     case <- ch:
3         return
4     default:
5         getMessages()
6         time.Sleep(1*time.Second)
7 }
```

Využili sme uzavretie komunikačného kanála. Teda ak je klient pripojený do komunikačnej miestnosti, má otvorený komunikačný kanál. Preto kým je tento komunikačný kanál otvorený, tak periodicky klient žiada o nové správy z komunikačnej miestnosti. Ak opustí komunikačnú miestnosť, kanál sa uzavrie. Uzav-

retím sa pošle správa cez komunikačný kanál a príkaz select vykoná vetvu, ktorá ukončí funkciu periodického žiadania o nové správy z komunikačnej miestnosti.

9 Záver

Cieľom bakalárskej práce bolo priblížiť princípy paralelného a súbežného programovania v jazyku Go so zameraním na model *Communicating Sequential Processes* a opísať mechanizmy, ktoré jazyk Go poskytuje na tvorbu súbežných aplikácií. Súčasťou práce bolo aj navrhnuť a implementovať prototyp chat servera, na ktorom boli demonštrované princípy komunikácie prostredníctvom komunikačných kanálov, synchronizácie a správy súbežne vykonávaných úloh.

Stanovené ciele práce boli splnené. V teoretickej časti boli opísané základné koncepty modelu CSP, goroutiny, komunikačné kanály a synchronizačné mechanizmy využívané v jazyku Go. Praktická časť preukázala ich využitie pri implementácii sieťovej aplikácie schopnej súbežne obsluhovať viacerých klientov. Návrh paralelných algoritmov spolu s dôkazmi ich správnosti zároveň ukázal možnosti vytvárania korektných riešení eliminujúcich problémy súbežného programovania.

Realizovaný prototyp potvrdil, že jazyk Go poskytuje prehľadné a efektívne prostriedky na vývoj súbežných sieťových aplikácií. Vďaka mechanizmom založeným na modeli CSP je možné navrhovať riešenia, ktoré sú zrozumiteľné, bezpečné z hľadiska komunikácie medzi goroutinami a zároveň jednoducho rozšíriteľné.

V budúcnosti by bolo možné implementované riešenie rozšíriť o databázové úložisko, autentifikáciu používateľov, šifrovanú komunikáciu alebo podporu distribuovaného spracovania správ.

10 Conclusion

The aim of this bachelor's thesis was to introduce the principles of parallel and concurrent programming in the Go programming language, with a focus on the *Communicating Sequential Processes* (CSP) model, and to describe the mechanisms that Go provides for developing concurrent applications. The thesis also included the design and implementation of a prototype chat server demonstrating the principles of communication through channels, synchronization, and the management of concurrently executing tasks.

The objectives of the thesis were successfully achieved. The theoretical part described the fundamental concepts of the CSP model, goroutines, communication channels, and synchronization mechanisms used in the Go programming language. The practical part demonstrated their application in the implementation of a network application capable of concurrently serving multiple clients. The design of parallel algorithms, together with proofs of their correctness, further demonstrated approaches to developing correct solutions that eliminate common concurrency-related issues.

The implemented prototype confirmed that the Go programming language provides clear and efficient tools for developing concurrent network applications. Thanks to the mechanisms based on the CSP model, it is possible to design solutions that are understandable, safe in terms of communication between goroutines, and easily extensible.

In the future, the implemented solution could be extended by adding persistent database storage, user authentication, encrypted communication, or support for distributed message processing.

Literatúra

- [1] Hoare, C. A. R. *Communicating Sequential Processes*. Englewood Cliffs, NJ: Prentice-Hall International, 1978.
- [2] Hoare, C. A. R. Communicating Sequential Processes. In Graham, S. L.; Rivest, R. L. (ed.). *Programming Techniques*. New York: Academic Press, 1978.
- [3] Baugh, John P. *Go Programming*. 2010. ISBN 9781453636671.
- [4] Beckingsale, David. *Towards Scalable Adaptive Mesh Refinement on Future Parallel Architectures*. 2015.
- [5] Cox-Buday, Katherine. *Concurrency in Go*. First. Sebastopol, CA: O'Reilly Media, 2017. 224 str. ISBN 978-1-491-94119-5.
- [6] Go Authors. *Go Runtime Source Code: runtime/chan.go*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02.
- [7] Go Authors. *Go Runtime Source Code: runtime/select.go*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02.